

Beyond Your Current Stack

A map of the software ecosystem — and how to learn it
without learning everything.

The Software World Is Bigger Day by Day

"There is a lot more out there than what I currently use."

FRONTEND / INTERFACE

React, Vue, Svelte
Swift, Kotlin, Flutter
CSS is a real language

SYSTEMS / BACKEND

Go, Rust, Elixir, Java
gRPC, REST, GraphQL
Auth, sessions, OAuth

DATA / INTELLIGENCE

Postgres, SQLite, DuckDB
Pandas, PyTorch
Embeddings, vector search

PLATFORM

Docker, Kubernetes
AWS, GCP, Terraform
CI/CD, observability

FOUNDATION

Linux, networking, TLS
Git and code review
Testing, profiling

WORKFLOW

Editors, terminals, GitHub
Feature flags, rollouts
Docs your team reads

You don't need to know all of this. You need to know it exists.

Every Piece Existed Because Someone Had a Problem

INTERFACE

The screen

React, SwiftUI, Figma

API

How systems talk

REST, gRPC, GraphQL

LOAD BALANCER

Spread traffic

Nginx, Envoy

CDN

Serve from nearby

Cloudflare, Fastly

DATABASE

Truth that survives

Postgres, SQLite

CACHE

Don't ask twice

Redis, in-process

MESSAGE QUEUE

Decouple slow work

SQS, Kafka, NATS

WORKER

Do it elsewhere

Celery, Temporal

OBJECT STORE

Files, not rows

S3, R2

CI / CD

Ship without fear

GitHub Actions

MONITORING

Know it broke

OpenTelemetry, Grafana

AUTH

Who is this?

OAuth, OIDC

Highlighted: the two you almost never need on day one.

Architecture Grows From Problems

PHASE 01

One service. One database.
It works.



PHASE 02

Reads are slow. p95 is
800ms.



PHASE 03

Emails block the request.
Users wait.



Every box you add traces back to a sentence like the orange ones —

Complexity should have a reason.

Don't Use Everything

POPULARITY DRIVEN

"This technology is trending, so we should use it."

PROBLEM DRIVEN

"We have this problem. Which solutions exist, and what are the trade-offs?"

THE SAME PROJECT, THREE MONTHS LATER

"We should move to Kubernetes."

You had one service. Now you run a control plane.

"We need real-time. Add Kafka."

You had 50 events/day. Now you have 3 new services.

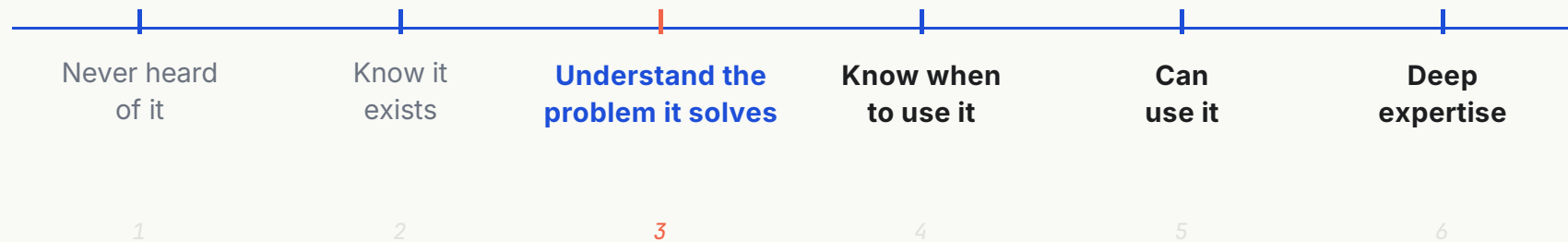
"The DB is slow. Add a cache."

You measured it. 800ms → 40ms.
You wrote why.

Know the toolbox. Choose the right tool.

Two of these three projects are still running. Nobody remembers why.

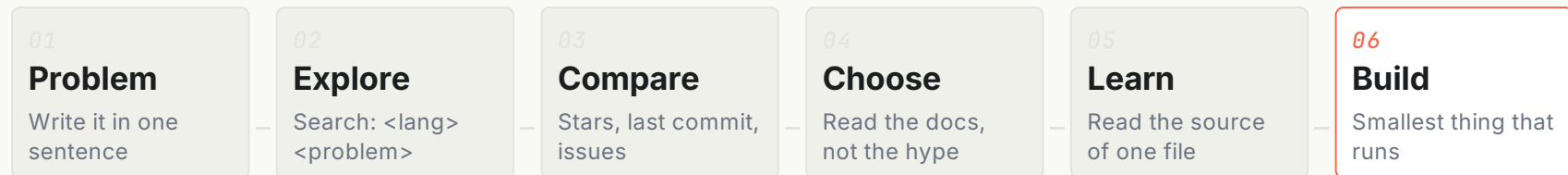
Know Enough to Know When to Learn More



You don't need to master everything. Rung 3 is where most people live.

You can read a job posting, hold the conversation, and know when to reach for a tool — without owning it.

When You Don't Know, Go Looking



- Has someone already solved this?
- What are the alternatives?
- What tools exist for this?
- Why do people actually use this?

Most of this takes ten minutes. Skipping it costs a week.

Step 03 is three numbers: stars, last commit date, open issue count.

GitHub Is a Map

<i>README.md</i>	why it exists
<i>/src</i>	how it's built
<i>/docs</i>	what it can do
<i>Issues</i>	what's broken
<i>Pull Requests</i>	why they chose it
<i>Releases</i>	how it matures

FOUR REPOS WORTH AN HOUR EACH

[torvalds/linux](#)

How a huge system stays navigable

[pallets/flask](#)

Small enough to read end to end

[excalidraw/excalidraw](#)

A real product, real trade-offs

[sindresorhus/awesome](#)

How great lists get curated

Instead of asking **"How do I build this?"**

Ask **"Who has already built this?"**

Learn From Software You Didn't Write

01 Start with the README

What problem does this claim to solve?

02 Read the directory tree

Where does the code actually live?

03 Find one real PR

What did they reject, and why?

04 Skim the test suite

What do they consider worth checking?

WHAT YOU LEARN WITHOUT CONTRIBUTING

- Architecture that survived contact
- Naming from people who write daily
- Test strategy in practice
- Docs written for the next person
- How trade-offs get argued
- What maintainers actually care about

You don't have to contribute to open source to learn from open source.

Start with a repo you already use every day — you'll have questions about it anyway.

The Ecosystem Has Doors

COMMUNITIES

Meetups & local groups

In person, every week

[Hacker News / lobste.rs](#)

Daily, high signal

[dev.to](#)

Write to learn

BUILDING

[Hackathons](#)

A weekend, real users

[Open source issues](#)

up-for-grabs.net

Side projects

The fastest teacher

OPPORTUNITIES

[GitHub Sponsors](#)

Fund what you use

[OSS curriculum](#)

Structured, free

Conferences

The hallway track

The links on this slide are live — open them from the deck.

Don't Collect. Participate.

COLLECTING (PASSIVE)

- 43 browser tabs open
- A course you started
- Bookmarked, never read
- Tutorials half-watched
- A Discord you muted

PARTICIPATING (ACTIVE)

- ✓ Clone a repo and run it
- ✓ Fix one typo in a README
- ✓ Open your first issue
- ✓ Ship something unused
- ✓ Ask a question in public

One thing you actually tried is worth more than a hundred things you bookmarked.

START HERE
up-for-grabs.net

This weekend: pick one item from the right column. Just one.

Build Something You Don't Normally Build

EXPLORE → BUILD → LEARN → REPEAT

Use the hackathon as a laboratory. The goal isn't to build — it's to find out what else exists.

> *go build*

Three questions worth leaving with: what will you read this week · what will you build · what will you finish?